



## Heterogeneous Computing, Übungsblatt 2, Sommer 2026

### *Aufgabe 1: Das SIMT-Ausführungsmodell und Warp-Divergenz (rechenintensiv)*

Ziel: sichtbar machen, dass eine GPU sehr viele Threads im Gleichschritt (SIMT) abarbeitet und dass datenabhängige Verzweigungen innerhalb eines Warps teuer werden, während sie eine CPU kaum stören. Implementieren Sie dazu einen geeigneten, rechenintensiven Kernel mit minimalem Speicherbedarf: jeder Thread bearbeitet genau ein Element und führt darauf eine konfigurierbare, rein arithmetische Last aus (z. B. eine iterierte Polynom-/Newton-Berechnung oder ein synthetisches „ALU-Mischen“ über  $k$  Iterationen – ohne nennenswerte Speicherzugriffe). Skalieren Sie die Problemgröße  $n$  bis in die Millionen. Messen Sie den Durchsatz Ihres Kernels (z. B. Operationen/s bzw. GFLOP/s) und stellen Sie die Skalierung über  $n$  dar. Halten Sie fest, ab welcher Problemgröße die GPU „warmläuft“ und wie viele Threads Sie starten müssen, um das Gerät auszulasten – ein Aufwand, der bei einer CPU-Lösung in dieser Form gar nicht erst anfällt. Führen Sie nun gezielt Warp-Divergenz ein: lassen Sie den Arbeitsaufwand pro Thread datenabhängig variieren (z. B. Verzweigung über `threadIdx % k` oder eine vom Element abhängige Schleifenlänge). Variieren Sie den Divergenzgrad systematisch von „keine Divergenz“ bis „jeder Thread im Warp nimmt einen anderen Pfad“ und messen Sie den Einbruch bei der Gesamtperformanz. Beschäftigen Sie sich mit der Frage: Warum serialisiert ein Warp divergente Pfade, und warum ist dieselbe Verzweigung auf der CPU (Sprungvorhersage, unabhängige Kerne) nahezu kostenlos? Genau hier wird der SIMT-Unterschied zu einer „normalen“ CPU sichtbar.

### *Aufgabe 2: Speicherzugriff: Latenz verstecken und Bandbreite (speicherintensiv)*

Ziel: sichtbar machen, dass die GPU Speicherlatenz nicht durch große Caches sondern durch sehr viele gleichzeitig lauffähige Warps versteckt – und dass das Zugriffsmuster über die tatsächlich erreichte Bandbreite entscheidet. Implementieren Sie einen speicherintensiven Kernel mit geringer arithmetischer Intensität (z. B. Streaming à la  $b[i] = a[i] * c$ ). Vergleichen Sie die folgenden drei Zugriffsmuster auf den Gerätespeicher: (1) zusammenhängend / coalesced (Stride 1), (2) gestriped (Stride  $k$ ) und (3) zufällig (Gather). Messen Sie für jedes Muster die erreichte effektive Bandbreite (GB/s) und setzen Sie sie ins Verhältnis zur maximal erreichbaren Bandbreite des Geräts (über `deviceQuery` o.ä. abfragbar). Zeigen Sie das Verstecken der Latenz über die Occupancy: variieren Sie die Blockgröße bzw. die Anzahl gleichzeitig residenter Warps und zeigen Sie, wie der Durchsatz davon abhängt, dass genügend Warps bereitstehen, um Speicher-Stalls zu überbrücken. Geben Sie die gemessene Occupancy an (Occupancy = aktive Warps / maximal mögliche aktive Warps). Erklären Sie den Unterschied der beiden Latenz-Strategien: Die GPU versteckt Latenz über massive Parallelität (viele Warps), während die CPU Latenzen über die Cache-Hierarchie und Prefetching verbirgt (sequenziell schnell, zufällig durch Cache-Misses langsam). Welche Konsequenzen hat dieser grundsätzlich andere Mechanismus im Hinblick auf eine effiziente Anordnung von Daten in GPU-Anwendungen?

### *Abgabe*

Abzugeben ist ein Repository mit Ihrer Implementierung als Pull-Request gegen das Github-Repository [2026S-HC](#). In diesem Repository soll sich auch ein kurzer Ergebnisbericht befinden. Vermeiden Sie jegliche personenbezogenen Informationen (also keine Matrikelnummer, kein Name), damit am Ende der Vorlesung alle Lösungen in diesem Repository zusammengefasst und veröffentlicht werden können. Legen Sie innerhalb des Repositories einen eigenen Unterordner an, in dem sich alle Lösungen zu diesem Aufgabenblatt inkl. dem Bericht befinden; das macht das Zusammenführen aller Abgaben etwas leichter. Außerdem müssen Sie in der Übung Ihre Realisierung in einem ca. 10-minütigen Vortrag vorstellen.

Abgabefrist ist der 12.7.2026.